COMPTE RENDU D'ANALYSE FORENSIQUE

CyberTalents CTF

Digital Forensics – History 102

Analyse d'une image AD1, d'un historique Edge et d'une donnée du registre Windows

Réalisé par : LAAROUSSI SALAH-EDDINE

Encadrante : Ikram Benabdelouahab

Plateforme : CyberTalents

Challenge : History 102

Catégorie : Digital Forensics – Hard (200 points)

Compte rendu des étapes, commandes, preuves et résultats obtenus

Novembre 2026

Table des matières

1 Introduction **2**

2 Environnement et outils utilisés **2**

3 Résolution du challenge **2**

3.1 Préparation et extraction de l'archive 2

3.2 Identification de l'image AD1 3

3.3 Recherche des fichiers pertinents et extraction 4

3.4 Calcul et vérification de l'empreinte MD5 5

3.5 Analyse de l'historique de Microsoft Edge 5

3.6 Analyse statique de l'exécutable 6

3.7 Lecture de la ruche SOFTWARE 6

3.8 Décodage de la valeur cachée par XOR 7

4 Flag obtenu et validation **8**

5 Bilan et conclusion **8**

6 Profil CyberTalents et ressources **8**

1. Introduction

Le challenge **History 102**, proposé sur la plateforme CyberTalents dans la catégorie *Digital Forensics*, décrit le cas d'un utilisateur ayant recherché une version crackée d'un logiciel. L'objectif est d'examiner des traces numériques pour retrouver le fichier suspect, l'adresse IP associée au téléchargement et une chaîne représentant une adresse de portefeuille virtuel.

Le format demandé est :

```
Flag{MD5_malicious_file_IP_address_from_site_ad_Wallet_address}
```

La démarche retenue repose sur **l'analyse hors ligne** : inspection de l'image forensique, extraction des artefacts, interrogation d'une base SQLite, examen statique d'un exécutable PE et lecture d'une ruche du registre Windows. Aucun lancement du fichier suspect n'est nécessaire.



Figure 1 – Description officielle du challenge et format du flag demandé.

2. Environnement et outils utilisés

Les opérations ont été réalisées sur **Kali Linux**, dans un environnement de travail dédié. Les fichiers originaux ont été conservés séparément des données extraites.

Outil	Utilité dans le challenge
`7z`, `file`	Vérifier l'archive fournie et extraire l'image `.ad1`.
AD1-tools	Lire les métadonnées, afficher l'arborescence et extraire les fichiers de l'image.
`find`, `md5sum`	Localiser le fichier suspect et calculer son empreinte MD5.
`sqlite3`	Interroger la base d'historique du navigateur Microsoft Edge.
`strings`, `objdump`	Examiner statiquement l'exécutable Windows et son code assembleur.
`reglookup`	Lire une valeur du registre Windows à partir de la ruche `SOFTWARE`.
Python 3	Décoder la donnée encodée en UTF-16LE et transformée par XOR.

3. Résolution du challenge

3.1 Préparation et extraction de l'archive

Le fichier transmis par CyberTalents est une archive `History102.7z`. Nous avons d'abord listé son contenu, puis procédé à son extraction :

```
file History102.7z
7z l History102.7z
7z x History102.7z -oHistory102
find History102 -type f
```

L'archive contient `History102.ad1`, d'environ **531 Mio**. L'extension `.ad1` correspond à un conteneur d'image logique utilisé en investigation numérique ; il ne s'agit pas d'un programme à exécuter.

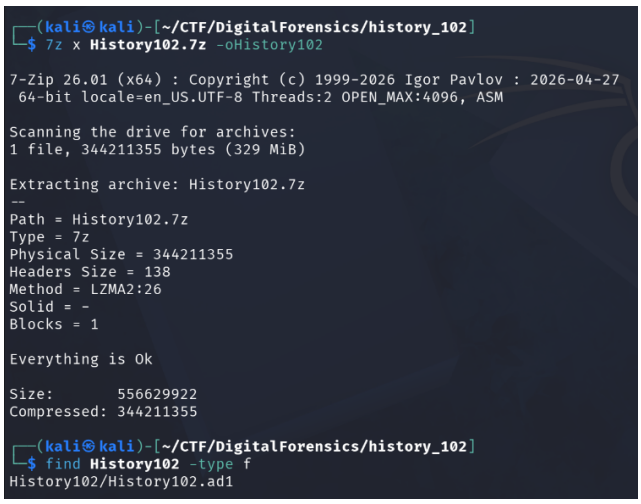


Figure 2 – Extraction réussie de l'archive contenant `History102.ad1`.

3.2 Identification de l'image AD1

La commande `file` renvoyait simplement `data` pour l'image ; cette détection n'était pas suffisante. Nous avons donc utilisé **AD1-tools**, compilé depuis son code source, pour lire la structure interne.

```
# Depuis le dossier du challenge :
./AD1-tools/AD1Tools/ad1info -i History102/History102.ad1
```

Les informations importantes sont :

```
Signature        : ADSEGMENTEDFILE
Signature        : ADLOGICALIMAGE
Version          : 4
Data source name : E:\NONAME [NTFS]
```

L'image contient donc des artefacts issus d'un volume Windows de type **NTFS**. La compilation de la version utilisée d'AD1-tools nécessitait les en-têtes de **FUSE 2** ; les en-têtes FUSE 3 ne sont pas interchangeables avec cette version.

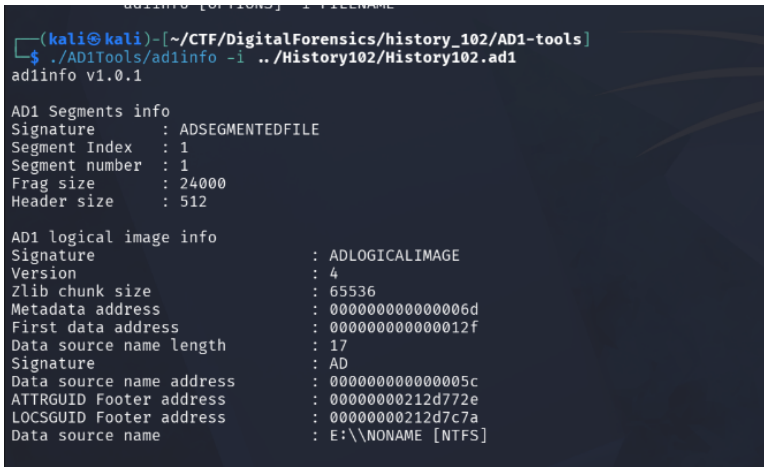


Figure 3 – Métadonnées de l'image logique AD1 : volume `E:\NONAME [NTFS]`.

3.3 Recherche des fichiers pertinents et extraction

L'option `-t` permet de parcourir l'arborescence contenue dans l'image :

```
./AD1-tools/AD1Tools/ad1info -t -i History102/History102.ad1
```

Dans le profil `Users/Gojo`, le répertoire `Downloads` contient les deux éléments suivants :

```
Users/Gojo/Downloads/AdobePhotoshopCrack.zip
Users/Gojo/Downloads/AdobePhotoshopCrack/AdobePhotoshopCrack.exe
```

Ces noms correspondent au scénario du challenge et désignent les principaux éléments à examiner. L'extraction complète des fichiers a été effectuée avec :

```
mkdir -p evidence
./AD1-tools/AD1Tools/ad1extract \
  -i History102/History102.ad1 -d evidence
```

Certaines métadonnées spécifiques à NTFS ont produit des messages d'erreur lors de l'extraction. Les deux fichiers recherchés étaient néanmoins présents dans le dossier d'extraction ; leur analyse a donc pu continuer.



Figure 4 – Arborescence `Users/Gojo/Downloads` et découverte du fichier suspect.

3.4 Calcul et vérification de l'empreinte MD5

La première composante du flag est le hachage MD5 du fichier malicieux. Nous avons calculé l'empreinte de `AdobePhotoshopCrack.exe` sans jamais l'exécuter :

```
EVID="$PWD/evidence/[root]"
EXE="$EVID/Users/Gojo/Downloads/AdobePhotoshopCrack/AdobePhotoshopCrack.exe"
md5sum "$EXE"
```

Résultat observé :

```
316fcaf0160b0025e13187624d0ac081
```

L'empreinte a été recoupée avec le contenu du fichier ZIP :

```
7z x -so "$EVID/Users/Gojo/Downloads/AdobePhotoshopCrack.zip" \
  AdobePhotoshopCrack.exe | md5sum
```

Le résultat est identique. Cette vérification confirme que l'exécutable extrait directement et celui contenu dans l'archive ZIP ont le même contenu.

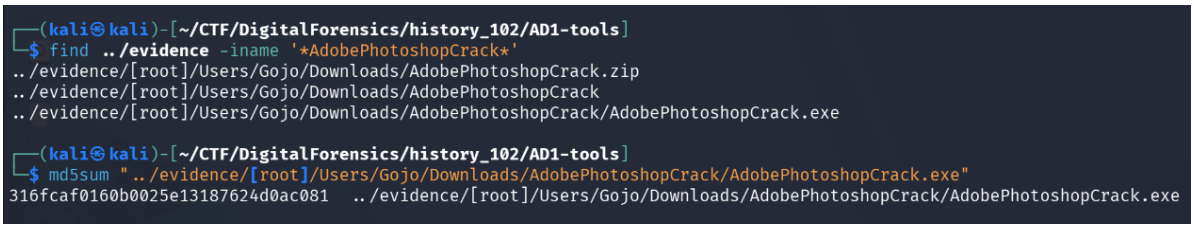


Figure 5 – Localisation de l'exécutable et empreinte MD5 calculée.

3.5 Analyse de l'historique de Microsoft Edge

Pour retrouver l'adresse IP, nous avons localisé la base SQLite du navigateur :

```
HIST="$EVID/Users/Gojo/AppData/Local/Microsoft/Edge/User Data/Default/History"
```

Cette base contient notamment la table `urls`. Une requête ciblée sur les pages liées au téléchargement révèle :

```
sqlite3 "$HIST" "SELECT id, url, title FROM urls
 WHERE url LIKE '%156.26.173.114%';"
```

Le résultat observé est le suivant :

```
41|http://156.26.173.114/download/AdobePhotoshopCrack.rar|Download Photoshop free crack
```

L'adresse **156.26.173.114** est donc associée, dans l'historique du navigateur, à une page de téléchargement de crack Photoshop. D'autres entrées montrent plusieurs domaines de redirection publicitaire (notamment `f5h7j.monster` et `mo13.biz`).

Limite de l'observation : l'entrée `urls` ne dispose pas ici d'une visite correspondante dans la

table `visits`, et ne reconstitue donc pas à elle seule toute la chaîne de redirection. Néanmoins, cette IP est la valeur relevée dans l'artefact lié au téléchargement et retenue pour le flag validé.

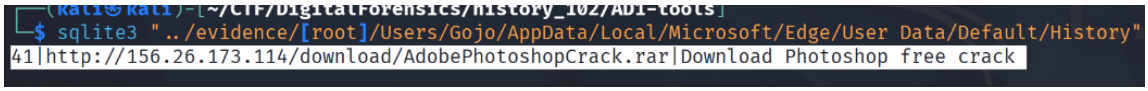


Figure 6 – Requête SQLite montrant l'adresse IP et le titre de la page de téléchargement.

3.6 Analyse statique de l'exécutable

Le programme est un **PE32+ pour Windows x86-64**. L'étude de ses chaînes de caractères fait ressortir deux indices situés à proximité dans le binaire :

```
strings -a -t x -n 4 "$EXE" | grep -Ei 'currentversion|secr3t'
```

```
2808 SOFTWARE\Microsoft\Windows NT\CurrentVersion
2835 secr3t
```

La présence des fonctions `RegOpenKeyExA` et `RegQueryValueExA` indique que l'exécutable est conçu pour accéder à une valeur du registre Windows. L'observation du code désassemblé révèle également une transformation par XOR :

```
objdump -d -M intel "$EXE" | grep -i 'xor    edx'
```

```
40176b: xor edx,0xffffff87
```

Sur un octet, la constante utilisée équivaut à **0x87**. Ce détail oriente vers la recherche d'une donnée dissimulée dans le registre, plutôt que vers une chaîne lisible directement avec `strings`.

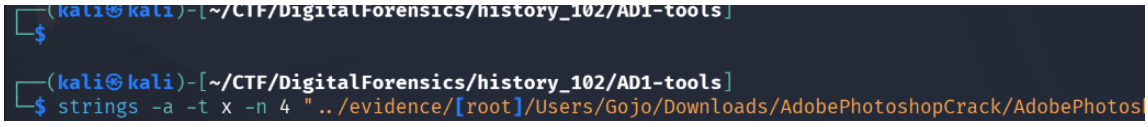


Figure 7 – Repérage du chemin de registre et de la valeur nommée `secr3t`.

3.7 Lecture de la ruche SOFTWARE

L'image extraite contient une copie de la ruche système :

```
HIVE="$EVID/Windows/System32/config/SOFTWARE"
reglookup -p '/Microsoft/Windows NT/CurrentVersion/secr3t' "$HIVE"
```

`reglookup` renvoie une entrée de type `SZ`, dont les octets apparaissent sous une forme encodée :

```
%D4%00%E2%00...
```

L'alternance avec des octets nuls correspond à un stockage de type UTF-16LE ; une simple lecture texte ne permet pas d'obtenir la valeur initiale. Nous devons extraire les octets utiles

puis appliquer le XOR identifié lors du désassemblage.

3.8 Décodage de la valeur cachée par XOR

Pour rendre l'analyse reproductible, le script suivant lit la valeur à l'aide de `reglookup`, décode les caractères `%XX`, récupère les octets de poids faible de l'UTF-16LE, puis applique la clé `0x87` :

```
import re
import subprocess
from urllib.parse import unquote_to_bytes

hive = "evidence/[root]/Windows/System32/config/SOFTWARE"
result = subprocess.run(
    ["reglookup", "-p",
     "/Microsoft/Windows NT/CurrentVersion/secr3t", hive],
    capture_output=True, text=True, check=True
)
match = re.search(r",SZ,([^,\n]+)", result.stdout)
if not match:
    raise SystemExit("Registry value not found")
raw = unquote_to_bytes(match.group(1))
message = bytes(b ^ 0x87 for b in raw[::2])
print(message.decode("latin-1").rstrip("\x00"))
```

Résultat du décodage :

```
Send_Money_To_bc1qxy2kgdygjrsqtzq2n0yrf2493!83kkfjhx0wlh
```

La chaîne commence par `Send_Money_To_` et contient l'élément attendu pour la composante *Wallet address*. Une particularité doit être conservée : `!` n'est pas un caractère valide pour une adresse Bitcoin Bech32. Une correction hypothétique basée sur le checksum ne correspondait pas au flag accepté. Pour ce challenge, **la valeur textuelle issue de l'artefact doit être reproduite exactement, y compris le caractère `!`.**



Figure 8 – Lecture de la valeur `secr3t` puis résultat du décodage XOR.

4. Flag obtenu et validation

Les trois composantes peuvent maintenant être réunies :

Composante	Valeur
MD5 du fichier	`316fcaf0160b0025e13187624d0ac081`
IP du site	`156.26.173.114`
Chaîne wallet	`bc1qxy2kgdygjrsqtzq2n0yrf2493!83kkfjhx0wlh`

Flag final :

```
Flag{316fcaf0160b0025e13187624d0ac081_156.26.173.114_bc1qxy2kgdygjrsqtzq2n0yrf2493!83
    kkfjhx0wlh}
```

Après soumission sur CyberTalents, la plateforme a affiché le message « **Congratulations ! Correct submission !** ». La résolution du challenge est ainsi confirmée par la capture d'écran ci-dessous.

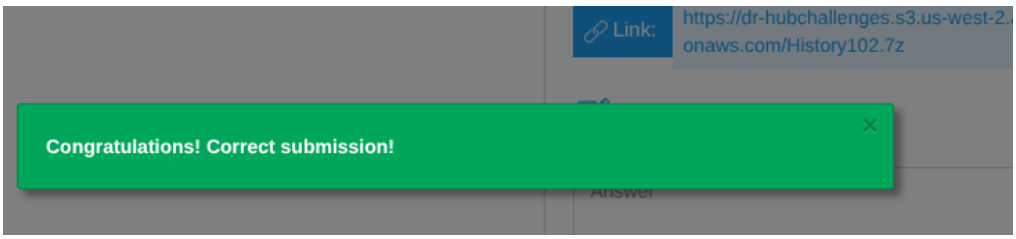


Figure 9 – Validation du flag sur CyberTalents : soumission acceptée.

5. Bilan et conclusion

Le challenge **History 102** montre l'intérêt de croiser plusieurs sources de preuves numériques. L'arborescence de l'image AD1 permet d'identifier le fichier concerné ; le MD5 caractérise son contenu ; l'historique de navigation fournit l'adresse IP et le contexte du téléchargement ; enfin, l'analyse statique de l'exécutable mène à une valeur cachée du registre Windows, décodée par XOR.

Deux enseignements sont particulièrement importants :

— Ne pas exécuter un binaire suspect lorsque les artefacts peuvent être examinés hors ligne.

— Distinguer la **donnée observée** de la **donnée corrigée par hypothèse** : dans ce cas, conserver le caractère `!` était indispensable pour reproduire le flag demandé.

Cette investigation a permis de mobiliser des compétences complémentaires en analyse de systèmes de fichiers, navigation web, reverse engineering et forensique du registre Windows.

6. Profil CyberTalents et ressources

Profil CyberTalents : `https://cybertalents.com/members/laaroussidin/profile`.

Outils utilisés : AD1-tools, `sqlite3`, `objdump`, `strings`, `reglookup` et Python 3.

Source des figures : captures d'écran de l'investigation, sélectionnées parmi les captures fournies. Seules les étapes significatives de la résolution sont présentées dans ce compte rendu.